

Senior FDE Architect Assessment of FDE-Final-Capstone

Executive summary

Verdict

Yes — the capstone has been delivered as a substantial synthetic academic FDE capstone. It has not been completed as a production-ready or fully externally assured system.

That distinction is important.

The repository itself makes the correct claim: it is a **synthetic academic Cell and Gene Therapy patient-to-batch orchestration demonstrator**, with all 21 AI FDE stages documented, but without implying owner approval, independent assurance, real-system deployment, regulatory validation or observed business benefits. Its current lifecycle decision is appropriately `RESTRICT_AND_CHANGE`, with AI disabled by default.

The supplied challenge brief asks for ten deliverables: brownfield assessment; domain/journey reconstruction; problem/KPIs; requirements; target architecture/ADRs/migration; three workflow POCs; TEVV; security/privacy/responsible-AI/governance; resilience/handover; and production-gap/90-day roadmap. It explicitly says **not every problem needs AI and deterministic engineering should be preferred where superior**.

Although your prompt says the “approach file” was not attached, the supplied `FDE_PDF(2).pdf` actually contains the more demanding **21-stage AI FDE Operating Model**, including the essential artefacts expected from mandate through engineering, assurance, operations, value proof and eventual retirement. I therefore used that PDF as the primary architectural maturity model, the challenge brief as the primary client acceptance contract, and the generic ML/data-science capstone categories only as a secondary cross-check. `filecite{turn0file0}`

My assessment

Measure	Assessment
Ten client deliverables represented	10/10
Weighted capstone completion	83%
Canonical artefacts marked <code>MISSING</code>	0
21 AI FDE stage folders represented	21/21
Automated repository tests	92 passed / 0 failed
Requirements internally verified	29/31 = 93.5%

Measure	Assessment
Requirements production verified	0/31
Evaluation catalogue	57 executed; 55 structural pass; 2 inconclusive
Original supplied cases fully property-graded	7/16
Original supplied cases partially property-graded	9/16
Extension cases structural-only	39
Registered full-dataset/concurrent NFR	Not verified
Real deployment	No
Independent assurance	No
Real business-value/ROI proof	No
Repository rating	8.2/10

The repository's own one-to-one operating-model register contains **185 artefact rows: 88** PRESENT_INTERNAL, **72** PARTIAL, **16** CONDITIONAL_NOT_SELECTED, **9** EXTERNAL_REQUIRED, and zero MISSING. Importantly, the register explicitly warns that these counts are not a completion score: a file existing is not the same thing as its associated gate having been approved.

My **83%** is therefore an architectural completion score rather than a filename-counting score.

What I would tell an assessor

“The capstone is delivered and technically demonstrable. It is a strong academic FDE implementation, not a production implementation. The remaining work is mostly evidence depth, external assurance, representative performance/integration proof and accountable sign-off — not creation of more documentation.”

That is also consistent with the repository's own acceptance matrix and progress tracker.

My biggest recommendation is therefore **do not add more architecture documents simply to increase apparent completeness**. The repository is already documentation-heavy. The next points come from execution:

1. deeper TEVV,
2. representative load testing,
3. accountable acceptance,
4. independent review,
5. stronger runtime/security evidence.

Those will improve the capstone far more than another twenty Markdown files.

Assessment basis and repository map

What the original ZIP establishes

I directly inspected and executed the attached:

```
AI_FDE_CGT_Patient_to_Batch_Orchestration.zip
```

I independently obtained:

```
SHA-256
74d31dc4694d52b0e9a9fb337e6ccda1086ad430b19287f9ca2b4b4e650ca979

Archive contents
132 files

Baseline verification
VERIFY_OK

Baseline tests
2 passed, 3 xfailed
```

That SHA exactly matches the original ZIP digest recorded by the final repository's progress tracker.

The attached ZIP is therefore not merely a random supporting archive. It is the actual brownfield/challenge baseline from which the final capstone was built. The final repository keeps a frozen `source_baseline` and its CI verifies the expected 132 extracted files and recorded source provenance. The README explains that GitHub Actions cannot receive the original external ZIP itself, so CI validates the recorded archive digest plus the individual frozen-file hashes.

The ZIP contains, among other things:

```
participant/
  CHALLENGE_BRIEF.md
  DISCOVERY_CHECKLIST.md

data/
  cgt_legacy.db
  manifest.json
  raw...

contracts/
  api_v1.yaml
  api_v2.yaml
  api_v3.yaml

evals/
  cases.csv
```

```

scenarios/
  inject_catalog.csv

shadow_ops/
  ...

docs/
  01_problem_context.md
  02_imperfection_layers.md
  03_architecture_current_state.md
  04_target_capabilities_not_prescribed_solutions.md
  05_data_map.md
  06_security_privacy_assurance.md

scripts/
tests/
tools/

```

That baseline is intentionally imperfect. Your final repository's Stage 2 work correctly treats it as a brownfield estate to discover rather than simply cleaning it and pretending the contradictions disappeared.

Final repository structure

The final repository is significantly more complete than the source ZIP. Its top-level layout includes source, tests, evidence, requirements, stage documentation, CI, Docker and verification tooling.

A practical architect's view is:

```

FDE-Final-Capstone/
├── .github/
│   └── workflows/
│       └── ci.yml
├── docs/
│   ├── 21_STAGE_ARTIFACT_REGISTER.csv
│   ├── 21_STAGE_ARTIFACT_REGISTER.md
│   ├── CLIENT_DELIVERABLE_ACCEPTANCE_MATRIX.md
│   ├── FINAL_CAPSTONE_REPORT.md
│   ├── CAPSTONE_PROGRESS_TRACKER.md
│   └── stages/
│       ├── stage_01/
│       ├── ...
│       └── stage_21/
├── evidence/
├── requirements/
│   ├── requirements.csv
│   ├── TRACEABILITY_MATRIX.csv
│   ├── verification_matrix.csv
│   └── verification_matrix.json

```

```

├─ reports/
├─ source_baseline/
├─ src/
│   └─ fde_capstone/
│       ├── adapters/
│       ├── services/
│       ├── web/
│       ├── api.py
│       ├── application.py
│       ├── cli.py
│       ├── demo.py
│       ├── evaluation.py
│       ├── source_cases.py
│       └─ storage.py
├─ tests/
│   ├── unit/integration/e2e-style coverage
│   ├── performance_test.py
│   └─ recovery_test.py
├─ tools/
├─ Dockerfile
├─ Makefile
├─ pyproject.toml
├─ pytest.ini
└─ README.md

```

The 21-stage documentation index gives a primary entry point for every operating-model stage from engagement charter through retirement/closure.

The implementation is real, not just documentation

The core application composes dedicated evidence, case, identity, readiness, command, quality and assistant services around the database. Live AI is deliberately prohibited; only `off` and deterministic `fake` modes are accepted.

For example, the code contains the architectural control:

```

provider = DeterministicAssistantFake() if ai_mode == "fake" else None

if ai_mode not in {"off", "fake"}:
    raise ValueError("LIVE_AI_PROVIDER_NOT_APPROVED")

```

See [application.py](#).

This is actually one of the stronger architectural decisions in the repository. The challenge explicitly says not to force AI into every problem, and the PRD states that AI must have **no consequential authority**.

The state layer is not a trivial dictionary demo either. [storage.py](#) provides persistent SQLite structures for evidence, assertions, domain events, audit, exceptions, identity proposals, commands, reservations, Quality evidence, recommendations and metrics. It separates `occurred_at` from

`recorded_at`, gives events evidence references, maintains a hash-linked audit chain, enables WAL mode and supports backup/state digest verification.

The slot command service also implements a meaningful distributed-systems concept rather than a toy happy path: payload-bound idempotency, replay detection, timeout-after-success as `OUTCOME_UNKNOWN`, reconciliation and compensation.

Likewise, the Quality path correctly refuses to equate manufacturing completion or inventory status with Quality release, models missing information as `UNKNOWN`, handles QC/deviation/thermal evidence and requires an authorised Quality decision.

CI and reproducibility are unusually good for a capstone

The repository has an actual GitHub Actions workflow triggered on push and pull request. It installs the project, executes pytest, generates JUnit evidence, rebuilds requirement verification, executes the evaluation catalogue, generates delivery evidence and runs the final capstone verifier.

The latest run on the assessed `main` commit completed successfully.

The latest commit assessed is:

```
91dfc5cded0641460bb572ced1660ccc5dda795f
Document 21-stage capstone and add source-backed demo evidence
```

The repository README reports 92 clean automated tests, 29/31 internally verified requirements and the limited TEVV results.

Runtime dependencies are deliberately small: Python 3.11+, FastAPI/Uvicorn for the API and pytest for development/testing.

The Docker image runs as a non-root UID, defaults AI to `off`, and launches the FastAPI application on port 8000.

That is strong capstone engineering discipline.

Capstone requirements crosswalk

Standard capstone components versus what actually exists

The table below applies your requested generic capstone taxonomy but does **not** punish the repository for intentionally rejecting unnecessary ML.

Expected component	State	Completeness	Repository evidence	Senior-architect judgement	Remediation
Problem statement	Yes	90%	Stage 3, final report, SCQA/root-cause/KPIs	Problem is well framed around unsafe fragmented patient-to-batch state and authority	Obtain KPI-owner confirmation and close supplied denominator/formula gaps
Requirements	Partial/strong	85%	<code>requirements.csv</code> , traceability, PRD; 29/31 internal verification	Better than most capstones; requirements-to-evidence discipline is strong	Owner approve PRD; close, waive two external-human requirements; verify key NFR
Architecture	Yes	90%	Target C4, ADRs, bounded AI design, migration plan	Strong target/current separation and authority design	Final owner approval; complete a few ADR rationales and target-vs-as-built API delta
Data pipeline	Partial	65%	Source-case reader, forensic/profile tooling, frozen baseline	Good provenance work, but not a governed full-estate ingestion/projection pipeline	Build one full-dataset ingestion + journey projection path
ETL / ELT	Partial	55%	Forensic/profile/build scripts; source reconstruction	Transformation logic exists as analysis/utilities, not an operational ETL product	Materialise raw→validated→projection stages with repeatable contracts
Data validation	Yes	90%	Hashes, data profiles, contracts, provenance, domain rules, tests	One of the strongest parts	Add schema/version failure tests to production-style ingestion
Feature engineering	N/A by design	Not scored	Deterministic orchestration selected	Do not manufacture features just to tick a rubric	Record explicit N/A decision in assessment crosswalk

Expected component	State	Completeness	Repository evidence	Senior-architect judgement	Remediation
Model training	N/A by design	Not scored	Live model prohibited; fake adapter only	Correct architectural choice for present problem	Train/evaluate a model only if a future use case passes AI suitability
Model/AI evaluation	Partial	70%	Fake assistant guardrails, 57-case evaluation catalogue	Boundary testing good; no live-model quality evidence, appropriately	If AI remains off, keep N/A; if AI enabled, provider/model eval becomes mandatory
System TEVV	Partial/strong	72%	92 tests; 57 eval cases; red team; recovery	Broad but too much is structural rather than property-complete	Finish critical property graders, human studies, registered load test
Deployment	Partial	60%	Docker, local Uvicorn, release manifests, simulated canary	Deployable locally; not deployed as service infrastructure	Add container build verification and one reproducible deployment environment
Monitoring	Partial	60%	metrics/audit, alert specs, monitoring simulations	Specification > operational telemetry	Add OpenTelemetry/structured metrics and dashboard evidence
Reproducibility	Yes	92%	frozen source, SHA verification, Makefile, Docker, CI, CLI	Excellent for academic work	Add dependency lock and immutable image/action pinning
Automated testing	Yes	90%	unit/integration/E2E-style tests, recovery, performance, CI	Strong suite, but pass count should not substitute for coverage depth	Add code coverage threshold, property tests and full registered performance test
Documentation	Yes	95%	21 stage folders, PRD, acceptance matrix, final report, roadmap	More than sufficient; risk is now over-documentation	Stop adding narrative; focus on executable evidence
CI	Yes	85%	<code>.github/workflows/ci.yml</code> , green latest run	Strong verification CI	Add lint/type/security/container/dependency gates; Docker build

Expected component	State	Completeness	Repository evidence	Senior-architect judgement	Remediation
CD / release automation	Partial	35%	release manifests and deployment simulation only	Correctly not presented as real CD	A reproducible staging deployment is enough for capstone; avoid unnecessary Kubernetes
Security/ privacy/ governance	Partial/ strong	80%	threat model, controls, OWASP mapping, RACI, system card	Good design evidence; real IAM/privacy/ Quality approvals absent	Independent security review + production auth only if progressing beyond academic demo
Final handover	Partial/ strong	80%	runbooks, SOP/ training docs, final report, roadmap	Academic handoff exists; no actual operational owner acceptance	Add named acceptance/ sign-off record and final defence package

The PRD itself is very candid: the browser and FastAPI surfaces are demonstrators, real identities/ signatures are absent, and the original 20-client latency condition remains open.

Why feature engineering and model training should not be added

This is an important point where I would challenge a mechanical “data-science capstone” checklist.

The supplied challenge explicitly instructs the participant to show where **deterministic engineering is superior to AI**.

The selected solution does exactly that:

- identity resolution requires evidence plus human authority;
- readiness is deterministic policy evaluation;
- reservation reliability is idempotency/reconciliation;
- Quality release is deterministic evidence and authority;
- AI is optional, recommendation-only and cannot mutate state.

Adding a trained model, vector database or notebook merely to make the capstone “look more AI” would make the design **worse**, not better.

The PDF’s operating model requires AI suitability analysis, model/RAG selection where appropriate and an AI-disabled path/failure strategy; it does not require inventing ML where the problem does not justify it. `filecite turn0file0`

The ten client deliverables

I used the challenge brief’s ten deliverables as the weighted basis for the overall 83% score.

Client deliverable	Status	Score	What is complete	What remains
Brownfield forensic assessment	Delivered	95%	Reproducible baseline, eight-lens assessment, inventory, source integrity	Real source-owner/field confirmation
Domain model + current-state journey reconstruction	Partial	75%	DDD/domain artefacts plus six source-backed journey examples	Full governed journey projection; controlled identity/COI/COC/ release adjudication
Problem statement + measurable KPIs	Partial/strong	75%	SCQA, root causes, baseline forensic metrics, KPI tree	Real business KPI formulas/denominators and measured outcome baseline
Functional/NFR/ assurance requirements	Partial/strong	85%	31 requirements, PRD, traceability, 29 internally verified	Human studies and registered performance NFR
Target architecture + ADRs + migration	Delivered as design	85%	C4 context/container/ component, ADRs, migration/coexistence strategy	Owner approval, production topology/ contracts, actual cutover evidence
Three workflow POCs	Delivered	95%	Identity, slot/ reconciliation, Quality/ release vertical slices	Representative source-system adapters and live integration
TEVV edge/ adversarial/outage/ authority	Partial	72%	92 tests, 57-case harness, red-team, recovery, simulations	Property-level depth, human factors, registered full load, independent TEVV
Security/privacy/ responsible AI/ governance	Delivered as analysis / partial proof	80%	Threat model, controls, OWASP, guardrails, AI-off, SBOM/AIBOM, governance	Real IAM, privacy/legal/ Quality review, independent security assurance
Resilience/ recovery/ operational handover	Partial	70%	Runbooks, local restore, monitoring, shadow/canary simulations	Representative DR, operator training/ adoption, actual operational ownership
Production-gap assessment + 90-day roadmap	Delivered	92%	Explicit gap matrix, CAPAs, 30/60/90 plan and stop gates	Sponsor funding/ ownership and execution of roadmap

This closely matches the repository's own architect acceptance matrix, which labels the same areas as delivered, partial or externally dependent.

The weighted calculation is:

Brownfield	10% × 95
Domain/journey	10% × 75
Problem/KPI	8% × 75
Requirements	10% × 85
Architecture/migration	12% × 85
Three POCs	15% × 95
TEVV	12% × 72
Security/governance	9% × 80
Operations/resilience	7% × 70
Production-gap/roadmap	7% × 92

Weighted completion	83.13%

Rounded overall completion: 83%.

Completion score and remaining gaps

What is genuinely complete

The following parts are sufficiently mature for a capstone defence.

Discovery and brownfield analysis are strong. The PDF's Stage 2 explicitly calls for process discovery, architecture discovery, brownfield assessment, C4, dependencies, data flows and trust boundaries. The repository has those artefacts and also uses the PDF's eight-lens legacy analysis pattern — imperfection, inconsistency, friction, complexity, volatility, uncertainty, hidden dependency and unknown unknown. The Stage 2 register marks most of this work internally present, with only the field-validated C4/topology still partial.

Domain and authority modelling are particularly good. The implementation does not silently unify conflicting identities. Identity proposals require corroborating evidence and an authorised decision bound to an exact proposal digest.

The command/reconciliation POC demonstrates an important real-world FDE concern. The system does not treat timeout as failure and blindly retry. A timeout after possible success becomes `OUTCOME_UNKNOWN`; reconciliation checks for the external effect, while partial effects can enter compensation.

The Quality POC has the right authority boundary. MES or ERP state cannot establish Quality release; QC, deviations and thermal evidence are evaluated, absence remains unknown, and release is performed through Quality authority.

Traceability is excellent. The project has machine-readable requirements and verification matrices plus the 185-row one-to-one operating-model artefact register.

Reproducibility is strong. Source provenance, tests, evaluation, requirement evidence and final verification are automated in CI, and the latest workflow completed successfully.

What is not complete

The most important unfinished technical item is not “AI”. It is the **full governed journey projection and its registered load test**.

Stage 15 explicitly records:

NOT_VERIFIED

for the required warmed, evidence-bearing journey projection across the fixed **27,507-row dataset**, under **20 concurrent clients**, with **p95 ≤ 250 ms** and correctness/provenance/error behaviour checked simultaneously. The existing six-patient explorer and micro-benchmark are explicitly not substitutes.

This is bigger than “run Locust for an hour”. The repository itself says the current app **does not yet have the specified fully ingested 27,507-row governed projection endpoint or 20-client runner**.

That is a real engineering gap.

TEVV depth is the second major gap

The Stage 15 report is honest:

- 92 automated tests passed;
- 57 evaluation cases were executed;
- 55 were structural passes;
- only 7 of 16 supplied original cases have full scoped property assertions;
- 9 of 16 are partial;
- 39 extension cases remain structural probes;
- 2 human-factor cases are inconclusive.

The phrase **“55 pass” must therefore never be presented as “55 fully validated scenarios”**.

The repo already warns against that exact mistake.

A senior evaluator will notice this immediately.

External assurance remains open

The PDF's Stage 15 output is an **“independently assured release candidate”**; Stage 16 expects an **“operationally and legally ready release”**; Stage 17 expects a controlled live service; Stage 18 requires actual operational resilience evidence; and Stage 19 expects observed business outcomes.

filecite turn0file0

The repository correctly says local simulation cannot establish those outcomes.

So this is the maturity boundary:

Stages 1–14
Mostly specified / built / internally evidenced

Stage 15
Internal assurance exists
Independent assurance does not

Stage 16
Local operational preparation exists
Legal/operational production readiness does not

Stage 17
Deployment simulation exists
Controlled live service does not

Stage 18
Monitoring/resilience simulation exists
Production observation does not

Stage 19
Technical feasibility evidence exists
Observed benefits/ROI do not

Stage 20
Correct decision = RESTRICT_AND_CHANGE

Stage 21
Local runtime retirement/reusable IP exists
Not enterprise system retirement

The repository's lifecycle decision is therefore architecturally correct rather than a failure.

API coverage is incomplete

The target C4 specifies a broader journey, identity, readiness, Quality, reconciliation and assistant surface, but the as-built FastAPI exposes only part of that target. The repository itself explicitly records this as an API coverage gap.

That is acceptable for a POC but should remain in the backlog rather than being presented as fully implemented target architecture.

Authentication is intentionally demo-grade

The API constructs a `Principal` from request headers such as the supplied principal, role and scope.

That is acceptable for synthetic demonstration but absolutely **not an enterprise identity boundary**.

The 90-day roadmap correctly identifies absence of enterprise IAM/MFA/separation-of-duties/e-signature as a production blocker and explicitly says demo headers are not identities.

Integrations remain simulations

Current adapters include a deterministic assistant fake and a slot simulator rather than real enterprise source connectors.

That means:

CRM	not integrated
Clinical	not integrated
MES	simulated / not integrated
LIMS	not integrated
QMS	not integrated
ERP	not integrated
Courier	not integrated
IdP	not integrated
Live LLM	deliberately not integrated

Again, that is legitimate for the academic POC, but it determines the ceiling of the completion score.

Production readiness is much lower than capstone completeness

I would separate these two numbers:

Academic/FDE capstone completeness: ~83%

Enterprise production readiness: ~40–45%

Do not combine them.

A capstone can be successful precisely because it discovers and explains why a system **should not yet go to production**.

The Stage 20 roadmap identifies twelve production-readiness gaps including mandate, controlled identity rules, milestone policy, Quality policy, IAM/e-signature, regulatory/privacy decisions, representative integrations, human factors, optional live-provider evaluation, full TEVV/load, measured business outcomes and independent assurance.

Repository rating

I rate the repository **8.2/10**.

Dimension	Rating
Brownfield discovery	9.0
Domain/authority architecture	9.0
Requirements/traceability	9.2
Implementation quality	8.5

Dimension	Rating
Data engineering maturity	7.2
TEVV depth	7.2
Security-by-design	8.0
Reproducibility/CI	9.0
Documentation	9.5
Production operations/integration	4.5
Overall repo quality	8.2/10

Why not 9+?

Because the next level must be earned through **execution evidence**, not more Markdown:

- full journey projection;
- registered performance run;
- deeper scenario assertions;
- independent review;
- human-factor evidence;
- real or representative contracts;
- stronger runtime authentication;
- production-style observability/DR.

Why not below 8?

Because most capstones do not have:

- an immutable source baseline;
- requirement-to-test traceability;
- explicit AI-disabled architecture;
- domain authority separation;
- idempotent external-effect handling;
- recovery evidence;
- CI-generated evaluation artefacts;
- a 185-row operating-model register;
- an honest lifecycle decision.

That is substantial FDE work.

What must be delivered for successful completion

The key distinction: academic completion versus production closure

You should **not** attempt to close production-only requirements with fabricated evidence.

For academic completion, the correct outcome can be:

“The capstone has successfully demonstrated the required engineering capabilities and has identified and controlled the remaining production gaps.”

It does **not** need to say:

“The system is production ready.”

The repository itself correctly prohibits the latter.

Minimum final submission package

I would require the following artefacts before I sign this off as a **successfully completed senior-FDE capstone**.

Acceptance and scope

- ☐ Immutable final Git commit/tag identified.
- ☐ Capstone Owner acceptance record signed or explicitly marked as academically accepted.
- ☐ Ten-deliverable acceptance matrix reviewed.
- ☐ Scope states **synthetic academic POC**.
- ☐ `RESTRICT_AND_CHANGE` lifecycle decision retained.
- ☐ Production, clinical, regulatory, validation and ROI claims explicitly prohibited.

The existing acceptance matrix and lifecycle documentation already provide nearly all of this, but the PRD still says **Capstone Owner review pending**.

Remaining effort: approximately 2–4 hours once the accountable reviewer is available.

Problem and discovery pack

- ☒ Brownfield forensic report.
- ☒ Current-state process/system architecture.
- ☒ Eight-lens brownfield assessment.
- ☒ Source inventory and provenance.
- ☒ Root-cause analysis.
- ☒ Baseline KPIs and KPI tree.
- ☐ Explicit statement of which business KPI formula gaps remain externally unresolved.

Most of this is already complete enough for academic assessment.

Domain/data pack

- ☒ Ubiquitous language/glossary.
- ☒ Bounded contexts/context map.
- ☒ Business rules/decision models.
- ☒ Domain events/state machines.
- ☒ Ownership/authority mapping.
- ☒ Data inventory.
- ☒ Quality profile.
- ☒ Lineage/provenance.
- ☒ Data gaps/datasheets.

- ☐ Clearly distinguish source reconstruction from adjudicated canonical journey.

The repository already does this distinction in its final report, particularly for the six source-backed journeys.

Architecture and requirements pack

- ☒ Final PRD.
- ☒ Functional requirements.
- ☒ NFRs.
- ☒ Assurance requirements.
- ☒ C4 Context view.
- ☒ C4 Container view.
- ☒ C4 Component view.
- ☒ API/data contracts.
- ☒ ADR register.
- ☒ AI suitability/non-AI decision.
- ☒ Threat model.
- ☒ Migration/coexistence/rollback strategy.
- ☒ Requirements → code → test → demo trace.
- ☐ Owner sign-off on the target baseline.

The target C4 is explicit about which enterprise connections remain simulated.

Working engineering artefacts

- ☒ Three POCs.
- ☒ API.
- ☒ CLI.
- ☒ Browser Control Tower.
- ☒ Persistence.
- ☒ Evidence/audit model.
- ☒ Identity-authority workflow.
- ☒ Idempotency/reconciliation workflow.
- ☒ Quality evidence/release workflow.
- ☒ AI-off mode.
- ☒ Dockerfile.
- ☒ Makefile.
- ☒ CI.
- ☒ Automated tests.
- ☐ Final immutable release tag and demo evidence bundle.

The final report accurately describes the three implemented vertical journeys.

TEVV pack

This is where I would require the most work.

- ☒ Evaluation strategy.
- ☒ Frozen scenario catalogue.
- ☒ Automated test evidence.
- ☒ Red-team/adversarial tests.

- [x] Recovery test.
- [x] Deployment/monitoring simulations.
- [x] Honest property-coverage classifications.
- [] Fully grade all **critical P0/P1 original cases**.
- [] Upgrade the nine partially graded supplied cases where feasible.
- [] Convert the highest-risk extension probes into explicit property assertions.
- [] Execute or formally scope/waive the two controlled human-factor studies.
- [] Execute the registered journey load test, **or explicitly accept it as an open production-readiness gap in the academic completion record**.
- [] Obtain an independent architecture/security/Quality review or record that independent assurance is beyond the academic acceptance boundary.

The present TEVV report already exposes the gaps clearly.

Estimated effort:

TEVV closure task	Estimate
Upgrade critical property-level graders	12–24 h
Finish nine partial original cases	6–10 h
Controlled human-factor studies	8–16 h + participant scheduling
Independent review	8–16 reviewer hours
Full dataset/projection/load path	20–40 h
Final evidence consolidation	2–4 h

The full performance item is larger because the registered protocol explicitly says the required full-dataset projection endpoint and runner do not yet exist.

What I would make mandatory before final defence

For a **capstone**, not a production launch, my P0 list would be only:

1. **Capstone Owner academic acceptance**
2. **Green CI on the final immutable commit**
3. **Demonstrable three-POC end-to-end run**
4. **Requirements/traceability report**
5. **TEVV report with no misleading “55 fully passed” claim**
6. **Explicit disposition of the unverified load NFR**
7. **Explicit disposition of the two human-factor studies**
8. **Final `RESTRICT_AND_CHANGE` production-gap decision**
9. **Final demo recording or assessor-run demo**
10. **Executive defence deck / decision story**

Everything else can remain an **explicit production CAPA** if your academic assessor agrees with that boundary.

That is the lowest-maintenance, architecturally honest path to completion.

What I would *not* add before submission

I would **not** delay this capstone in order to add:

- Kubernetes;
- Terraform for a fictitious cloud environment;
- Kafka;
- a vector database;
- RAG;
- multi-agent orchestration;
- a trained ML model;
- notebooks;
- a microservice split;
- enterprise-grade data lake infrastructure.

None of those solves the present assessment gaps.

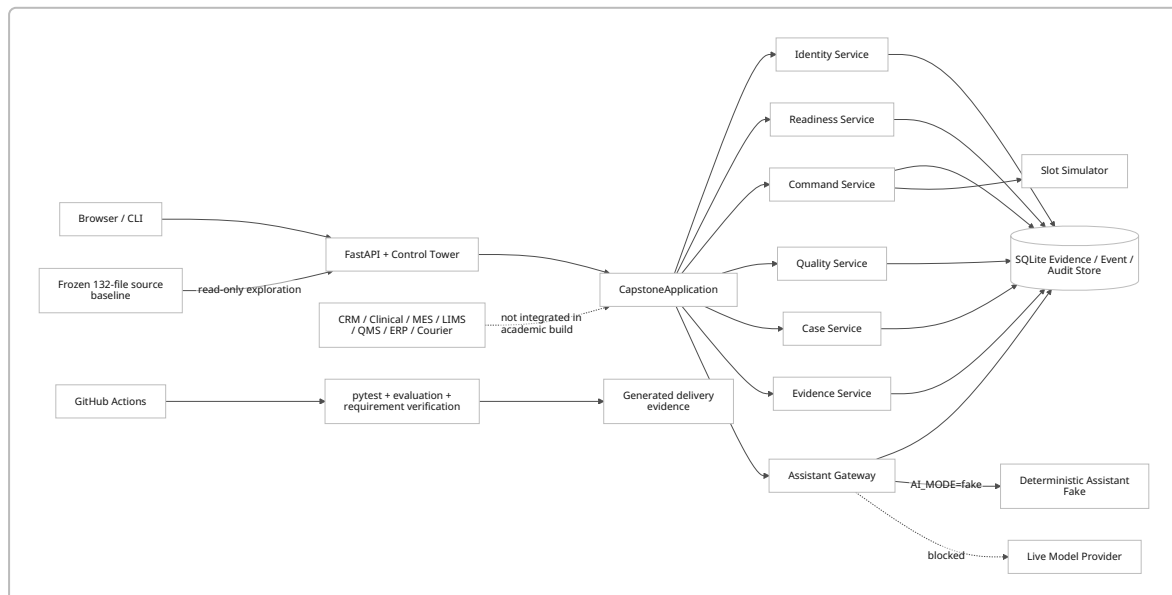
Your repository already demonstrates the more valuable judgement: **when not to use AI and when not to over-architect**. The challenge explicitly rewards that.

Architecture and engineering recommendations

Current as-built architecture

The repository's own target C4 is good.

I would describe the **actual academic system** to the assessor like this:



This matches the application's composition, source explorer, simulator adapters, bounded fake assistant and SQLite persistence.

Scalability

The biggest scalability limitation is not Python or FastAPI. It is the **single-process SQLite architecture plus absence of the fully materialised journey projection**.

SQLite with WAL and a process-local lock is an excellent POC choice and keeps the capstone reproducible.

For a production evolution, I would change the runtime boundary approximately as follows:

Synthetic POC	Production candidate
-----	-----
SQLite	PostgreSQL / managed relational store
Process lock	Database transactions/advisory locking
Slot simulator	Contracted MES/scheduler adapter
File fixture ingestion	Versioned source connectors / CDC
Local metrics table	OpenTelemetry + metrics backend
Demo HTTP headers	Enterprise OIDC/MFA/claims
Single process	Horizontally scalable stateless API
Local backup	Tested PITR / cross-zone recovery
Manual data contracts	Contract registry + compatibility checks

But I would **not do all of this before the capstone defence**.

First prove the complete journey projection and load requirement. That is the smallest executable scalability test that directly closes an existing rubric gap.

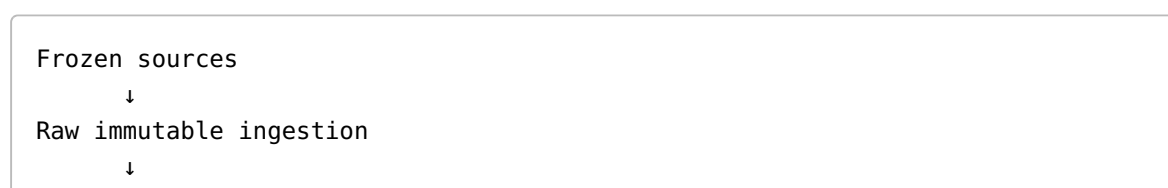
Data engineering

The data architecture has good conceptual ingredients:

- source-specific assertions;
- provenance;
- typed evidence;
- occurred-versus-recorded time;
- bitemporal-style known-at query support;
- event records;
- conflict preservation;
- explicit `UNKNOWN`;
- hash-linked audit.

What it lacks is a complete operational ingestion/projection lifecycle.

The next data increment should therefore be only:



```
Contract/schema validation
↓
Source-specific assertions
↓
Conflict-preserving normalisation
↓
Patient-to-batch journey projection
↓
Evidence-bearing API response
↓
20-client performance + correctness test
```

That one vertical pipeline would increase the engineering maturity more than adding a general-purpose warehouse.

Security

The threat model itself is good and enumerates six trust boundaries and twelve threats including spoofed roles, payload swapping, audit tampering, IDOR, prompt injection, model hallucination, leakage, outage, replay, corrupt source records, dependency compromise and secret leakage.

For the current synthetic scope, that is enough.

For anything beyond academic demonstration, the order should be:

Authentication first. Replace trusted demo headers with OIDC/OAuth2 identities and server-verified claims. The roadmap already identifies this as G-05.

Then authorisation. Preserve the current role/scope and separation-of-duties concepts, but bind them to authoritative enterprise identity rather than caller-provided header strings.

Then evidence/signature semantics. In a Quality context, authentication alone is not an electronic signature. Meaning, linkage, attribution and validation requirements need controlled decisions before real release authority exists. The roadmap already recognises this.

Then supply-chain hardening. The current Dockerfile is already non-root, which is good. I would additionally:

- pin the base image by digest;
- lock exact dependency versions;
- pin GitHub Actions to immutable commit SHAs;
- add `pip-audit` or equivalent;
- add static security scanning;
- scan the built container;
- generate/sign an SBOM during CI rather than relying only on a committed academic manifest.

CI and reproducibility

The existing pipeline is:

```
checkout
  ↓
Python 3.12
  ↓
install
  ↓
pytest + JUnit
  ↓
test result summary
  ↓
requirement verification
  ↓
evaluation catalogue
  ↓
delivery evidence
  ↓
final-capstone verification
```

That is a good evidence-oriented CI flow.

I would add a compact quality lane:

```
ruff
mypy/pyright
coverage gate
dependency audit
secret scan
Docker build
container scan
```

Do not create a complex enterprise pipeline.

The current capstone needs a stronger **gate**, not a larger platform.

Reproducibility gaps

The current dependency specification uses version ranges rather than a lock file.

I would add one of:

```
uv.lock
```

or

```
requirements.lock
```

Then ensure CI installs from the lock.

Similarly, `python:3.12-slim` is convenient but mutable; a final assessment build should pin a digest.

These are small changes with disproportionate reproducibility value.

Monitoring

The current system stores local metrics and verifies an audit chain, while the stage documents define telemetry and alert expectations.

For a stronger demonstration, add only a few externally visible measures:

```
journey_projection_latency_seconds
projection_unknown_total
source_conflict_total
authorization_denied_total
command_outcome_unknown_total
command_reconciliation_total
quality_release_denied_total
audit_chain_valid
```

Expose them as Prometheus/OpenTelemetry metrics, then capture one dashboard screenshot in the final evidence pack.

That is enough to demonstrate observability architecture without building a full operations platform.

Execution and closure timeline

Environment and commands

The repository targets Python 3.11+ and its CI uses Python 3.12.

A clean evaluator setup should be:

```
git clone https://github.com/86sunbot/FDE-Final-Capstone.git
cd FDE-Final-Capstone

python3 -m venv .venv
source .venv/bin/activate

python -m pip install -e '.[dev,api]'
```

Run the complete test suite:

```
python -m pytest -q
```

Run the evaluation harness:

```
PYTHONPATH=src \  
python -m fde_capstone.cli evaluate \  
  --db runtime/evaluation.db \  
  --output reports/generated/evaluation-results.json
```

Run the final repository verifier:

```
python tools/verify_final_capstone.py
```

The Makefile also exposes `test`, `legacy-test`, `demo`, `app`, `evaluate` and `verify` targets.

Start the browser application:

```
FDE_DB=runtime/control-tower.db \  
AI_MODE=off \  
PYTHONPATH=src \  
python -m uvicorn fde_capstone.api:app \  
  --host 127.0.0.1 \  
  --port 8000
```

Then visit:

```
http://127.0.0.1:8000  
http://127.0.0.1:8000/docs
```

These are the repository's documented demo commands.

Docker should also be demonstrated:

```
docker build -t fde-final-capstone:1.2.0 .  
  
docker run --rm \  
  -p 8000:8000 \  
  -e AI_MODE=off \  
  fde-final-capstone:1.2.0
```

The image uses a non-root runtime user and AI-off default.

What I could and could not execute during this review

I directly executed the **attached original ZIP baseline** and confirmed:

VERIFY_OK

2 passed, 3 xfailed

SHA-256 matches the final repository's recorded source archive hash

132 files in the supplied archive

I did **not** run the current private GitHub repository inside my local execution container because the connected GitHub repository is exposed through the GitHub connector rather than as a locally mounted Git working tree.

However, I inspected the current source through the GitHub connection and independently checked the latest GitHub Actions result. The current CI workflow completed successfully on the assessed main commit.

The repository's current README and generated reports record 92 passing tests.

Therefore:

Attached brownfield ZIP:

Directly executed in this review YES

Current GitHub source:

Source inspected YES

Current CI inspected YES

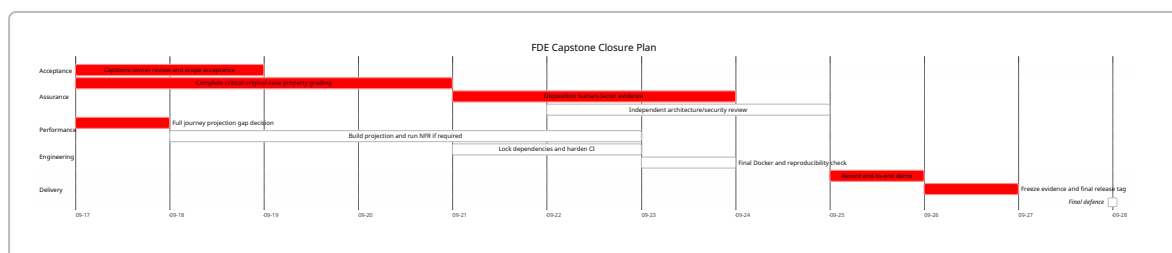
Locally cloned/executed by me NO

Latest GitHub Actions run successful YES

Practical remaining-work plan

I would not execute the repository's full 90-day **production** roadmap before submitting the academic capstone. That roadmap is correctly aimed at a future pilot and extends through December 2026.

For capstone closure, use a much smaller sequence:



The performance track is the only item that may materially extend the schedule because the repository currently lacks the required full projection endpoint, not merely the load runner.

Final student checklist

Before declaring “**Capstone Successfully Completed**”, I would require this exact checklist:

- ☐ Final commit is frozen and tagged.
- ☐ Latest CI is green.
- ☐ 92-test or successor suite result is attached as machine-readable JUnit evidence.
- ☐ Ten challenge deliverables are linked from one acceptance page.
- ☐ All 21 operating-model stages point to their canonical artefacts.
- ☐ PRD is reviewed by the Capstone Owner or explicitly academically accepted.
- ☐ Requirements → architecture → code → test → demo trace is intact.
- ☐ Three POCs run from a clean environment.
- ☐ Browser demo clearly distinguishes scripted POCs from frozen source-case exploration.
- ☐ Source ZIP digest and 132-file provenance are verified.
- ☐ No claim says 55 scenarios are fully validated.
- ☐ Critical P0/P1 properties have explicit pass/fail assertions.
- ☐ Human-factor cases are either executed or explicitly accepted as externally open.
- ☐ Registered 20-client/full-dataset NFR is either executed or explicitly accepted as an open production-readiness requirement.
- ☐ Security review states demo headers are not enterprise IAM.
- ☐ AI remains ☐ off by default.
- ☐ No live AI/provider capability is implied.
- ☐ Recovery evidence is attached and accurately labelled local/synthetic.
- ☐ Deployment evidence is accurately labelled simulation, not production.
- ☐ Before/after KPI and ROI claims remain unproven unless real evidence exists.
- ☐ Seven open CAPAs remain visible.
- ☐ ☐ RESTRICT_AND_CHANGE is retained until those external gates are satisfied.
- ☐ Final defence includes problem → evidence → architecture → POCs → TEVV → limitations → decision.
- ☐ Demo video or live assessor-run walkthrough is captured.
- ☐ Final report and evidence bundle are attached to an immutable release.

The strongest files to place directly in the assessor's path are:

[Repository README](#)

[Client Challenge Brief](#)

[Client Deliverable Acceptance Matrix](#)

[21-Stage Artifact Register](#)

[Final Capstone Report](#)

[Product Requirements Document](#)

[Target C4 Baseline and As-Built Delta](#)

[TEVV Report](#)

[Registered Load-Test Protocol](#)

[Threat Model](#)

[Production Gap and 90-Day Roadmap](#)

[CI Workflow](#)

The bottom-line architect decision is therefore:

The capstone is already defensible as a successful academic FDE delivery, provided its scope remains a synthetic engineering demonstrator. I would mark it “delivered with closure conditions”, not “unfinished”. Its 83% score reflects evidence maturity rather than missing basic functionality. To move it into the 90–95% range, stop writing more design documents and close the remaining high-value evidence gaps: owner acceptance, deeper TEVV, the full journey/load NFR, human-factor disposition and independent review. Production-readiness should remain explicitly out of scope until the Stage 20 CAPAs are actually closed.
